Cassandra — Plateforme d’observabilité intelligente

Point d’avancement de projet • Chanaz Jammoussi • Juillet 2026

CONTEXTE ET OBJECTIF

Ce projet de stage vise à concevoir et implémenter **Cassandra**, une plateforme d’observabilité intelligente dédiée à la détection automatique des dégradations de performance d’API. L’enjeu de départ est concret : les équipes opérationnelles s’appuient aujourd’hui majoritairement sur des seuils statiques pour déclencher leurs alertes. Cette approche, bien qu’immédiate à mettre en place, souffre de deux limites structurelles — un taux élevé de faux positifs lorsque les seuils sont trop sensibles, et des manques de détection lorsqu’ils sont trop permissifs — sans jamais fournir d’explication sur la cause probable d’un incident.

L’objectif de Cassandra est triple : **détecter** les dégradations de latence et de taux d’erreur par endpoint API, **les attribuer** à des événements de déploiement probables, et **les expliquer** en langage naturel pour faciliter le diagnostic. La thèse centrale du projet est quantitative : prouver, par une évaluation rigoureuse avec injection de fautes contrôlée, qu’une approche en couches (baseline saisonnière + détection non supervisée) offre de meilleures performances de détection que le seuillage statique, à coût de faux positifs maîtrisé.

ARCHITECTURE ET TECHNOLOGIES

La plateforme s’articule autour d’un pipeline en sept étapes, entièrement conteneurisé via Docker Compose pour en garantir la reproductibilité. Le trafic synthétique est généré par **k6** vers trois microservices de démonstration (gateway, orders, payments), instrumentés avec **OpenTelemetry**. Les traces OTLP sont acheminées vers un **OTel Collector** qui calcule en temps réel les métriques RED (Rate, Errors, Duration) via son connecteur *spanmetrics*, puis les expose à **Prometheus**. Un module de scraping interroge Prometheus toutes les 60 secondes pour alimenter les tables de features dans **TimescaleDB**, une base de données temporelle PostgreSQL haute performance. L’ensemble des données analytiques réside dans TimescaleDB, dont la compression native réduit l’empreinte d’un facteur environ 6 sur les données historiques. La visualisation est assurée par **Grafana**, les notifications sont envoyées vers **Slack**, et le projet compte 68 tests unitaires avec intégration continue sur GitHub Actions.

FONCTIONNALITÉS IMPLÉMENTÉES

Collecte et stockage des métriques. Le scraper extrait les percentiles de latence (p50, p95, p99), le taux d’erreurs et le débit par endpoint depuis Prometheus, puis les persiste dans TimescaleDB.

Construction automatique d'une baseline saisonnière. Un job de baseline calcule, pour chaque couple (endpoint, jour de la semaine × heure), les quantiles p10/p50/p90 à partir de l'historique, avec un mécanisme de cold-start pour les endpoints récents.

Détection en couches. C'est le composant central du projet, combinant trois couches indépendantes :

- *Layer 0 (statique)* : déclenchement immédiat si une métrique dépasse un SLO prédéfini ;
- *Layer 1 (baseline)* : comparaison avec la bande saisonnière attendue, produisant un score de déviation normalisé et une **direction** (dégradation vs amélioration) ;
- *Layer 2 (Isolation Forest)* : détection non supervisée multivariée sur des features endpoint-relatives (déviations de baseline, ratio p99/p50, pente Theil-Sen, delta de débit). La contribution de cette couche est **conditionnée par la direction** issue de la couche 1, évitant qu'une performance anormalement bonne ne déclenche une alerte.

Le score combiné est un blend calibré des couches 1 et 2, avec un seuil de déclenchement fixé à 0,60, déterminé empiriquement comme le point de F1 maximal.

Machine à états d'alerte. Les alertes suivent un cycle de vie à cinq états (OK → Pending → Firing → Resolving → OK), avec hystérésis montante et descendante pour prévenir le flapping. Seule la transition vers l'état Firing déclenche la corrélation et l'explication LLM, ce qui maîtrise le coût opérationnel et le bruit.

Corrélation avec les déploiements. Un registre de déploiements (exposé via une API REST) permet d'associer temporellement chaque dégradation détectée à un événement récent dans une fenêtre causale de 30 minutes, avec un score de corrélation.

Explication par LLM. Lorsqu'une alerte passe en Firing, le moteur d'explication soumet le contexte structuré de l'anomalie à un grand modèle de langage (Google Gemini), qui génère un résumé d'incident en langage naturel : cause suspectée, écart mesuré par rapport à la baseline, top-3 des features contributrices, et vérifications recommandées. Un mécanisme de fallback basé sur des templates garantit la continuité en cas d'indisponibilité du LLM.

Cycle de vie du modèle ML. Un job nightly réentraîne l'Isolation Forest sur l'historique glissant, en excluant strictement les fenêtres d'injection de fautes (jeu de test). Le nouveau modèle n'est promu qu'après validation par une *sanity gate* vérifiant l'absence de dérive de la distribution des scores. Les artefacts sont versionnés par horodatage.

Injection de fautes et évaluation. Cinq types de pannes contrôlées ont été développés (dégradation progressive, échelon de latence, burst d'erreurs, propagation downstream, contention de pool). Chaque injection constitue une étiquette de vérité terrain exploitée par le module d'évaluation, qui mesure précision, rappel, F1-score, taux de faux positifs par heure et délai de détection médian, en comparaison directe entre l'approche statique et l'approche en couches.

RÉSULTATS OBTENUS

L'évaluation menée sur une campagne de 31 injections réparties en 20 scénarios établit les chiffres suivants :

Métrique	Seuillage statique (Layer 0)	Détecteur en couches
Taux de détection global	65 %	71 %
Faux positifs / heure	1,13	1,29
Délai de détection médian	—	124 s

Le gain le plus net concerne les dégradations progressives par échelon : le taux de détection passe de 80 % à 100 %, pour un surcoût de faux positifs modeste de +0,16 FP/heure. Par ailleurs, la comparaison supervisé vs non supervisé (PR-AUC quasi identique entre Isolation Forest et gradient boosting) confirme que le choix non supervisé est pleinement justifié, sans exiger d'historique d'incidents labellisés. La qualité des explications LLM a été évaluée sur une rubrique à six critères (structure, cohérence des chiffres, actionnabilité, cadrage de l'incertitude) et obtient un score moyen de **5,00/6**. Enfin, l'attribution de déploiement identifie le bon commit en moins de cinq minutes avec un score de corrélation d'environ 0,9 sur le scénario de référence.

TRAVAUX EN COURS

Plusieurs axes sont actuellement en progression : approfondissement de l'analyse comparative entre l'approche statique et l'approche en couches, notamment pour mieux caractériser les conditions dans lesquelles le layered apporte le plus de valeur ; enrichissement du monitoring interne de la plateforme pour affiner la supervision de ses propres performances opérationnelles ; finalisation de la documentation technique ; et préparation de la démonstration finale bout-en-bout.

PROCHAINES ÉTAPES

À court terme : consolider le bilan d'évaluation avec des scénarios supplémentaires, explorer l'intégration de métriques de saturation comme features optionnelles pour améliorer la détection des fautes de contention subtiles, et finaliser la présentation de soutenance. À plus long terme, l'alerte précoce — estimation du délai avant dépassement du SLO par extrapolation de tendance — est identifiée comme un axe d'extension naturel, déjà partiellement implémenté à titre exploratoire.